CURRENCY CONVERTER USING PYTHON

MINI PROJECT REPORT

Submitted by

Srinidhi U

B.Sc. Information Technology
III Year

Department of Information Technology

Rathinam College of Liberal Arts and Science
TIPS Global, Kovilpalayam

Academic Year: 2026–2027

Project Technology: Python Programming

Project Type: Mini Project

ABSTRACT

The Currency Converter is a simple Python-based mini project designed to convert an amount from one currency to another. This application supports three currencies: USD (United States Dollar), EUR (Euro), and CAD (Canadian Dollar).

The program accepts an amount from the user and asks for the source currency and target currency. It uses predefined exchange rates stored in a Python dictionary to calculate the converted amount.

The program uses while loops to repeatedly request valid input and exception handling to manage invalid numerical values. Conditional statements are used to check whether the selected currencies are the same or different.

This project demonstrates the practical application of Python programming concepts such as loops, dictionaries, conditional statements, type conversion, and exception handling. It is suitable for beginners who want to improve their programming and problem-solving skills.

INTRODUCTION

Currency conversion is the process of converting the value of one country's currency into another country's currency. It is commonly used in international business, online shopping, tourism, banking, and financial transactions.

The Currency Converter project is developed using Python programming. It allows users to enter an amount, select a source currency, and select a target currency.

The program calculates the converted amount using predefined exchange rates. It also validates user input to prevent invalid amounts and unsupported currency codes.

This project provides practical knowledge of Python programming and demonstrates how dictionaries, loops, and exception handling can be used to develop a simple application.

OBJECTIVES

The main objectives of this project are:

To develop a simple currency conversion application using Python.

To convert amounts between USD, EUR, and CAD.

To validate numerical input entered by the user.

To handle invalid input using exception handling.

To use dictionaries to store predefined exchange rates.

To implement while loops for repeated input validation.

SOFT SKILLS:

Component	Specification
Programming Language	Python
IDE	Visual Studio Code
Operating System	Windows
Libraries	Python built-in features
Version Control	Git and GitHub (optional)

HARD SKILLS:

Component	Specification
Processor	Intel or equivalent
RAM	Minimum 4 GB
Storage	At least 100 MB free space
Input Device	Keyboard
Output Device	Monitor

CURRENCY USED IN THIS PROGRAM:

Currency Code	Currency Name	Country/Region
USD	United States Dollar	United States
EUR	Euro	Eurozone countries
CAD	Canadian Dollar	Canada

Dictionary

A dictionary stores information in key-value pairs.

Example:

```
exchangerates = {
    'USD': {'EUR': 0.85, 'CAD': 1.25},
    'EUR': {'USD': 1.18, 'CAD': 1.47},
    'CAD': {'USD': 0.80, 'EUR': 0.68}
}
```

Purpose: It stores the predefined exchange rates used for currency conversion.

ALGORITHM: CURRENCY CONVERTER

Step 1: Start the program.

Step 2: Ask the user to enter the amount to be converted.

Step 3: Validate the amount using a while loop and try-except block. Reject invalid, zero, and negative amounts.

Step 4: Define the supported currencies: USD, EUR, and CAD.

Step 5: Get the source currency and validate it.

Step 6: Get the target currency and validate it.

Step 7: Retrieve the appropriate exchange rate from the dictionary and calculate the converted amount. If both currencies are the same, retain the original amount.

Step 8: Display the converted amount and stop the program

SOURCE CODE: CURRENCY CONVERTER

```
while True:
```

```
    try:
        amount = float(input('Enter the amount: '))
        if amount <= 0:
            raise ValueError
        break
    except ValueError:
        print('Invalid amount')

currencies = ('USD', 'EUR', 'CAD')

while True:
    sourcecurrency = input(
        'Source currency (USD/EUR/CAD): '
    ).upper()

    if sourcecurrency not in currencies:
        print('Invalid currency')
    else:
        break

while True:
    targetcurrency = input(
        'Target currency (USD/EUR/CAD): '
    ).upper()

    if targetcurrency not in currencies:
        print('Invalid currency')
    else:
        break

exchangerates = {
    'USD': {'EUR': 0.85, 'CAD': 1.25},
    'EUR': {'USD': 1.18, 'CAD': 1.47},
    'CAD': {'USD': 0.80, 'EUR': 0.68}
}

if sourcecurrency == targetcurrency:
    convertedamount = amount
else:
    convertedamount = (
        amount * exchangerates[sourcecurrency][targetcurrency]
    )

print(
    f'{amount} {sourcecurrency} is equal to '
    f'{convertedamount:.2f} {targetcurrency}'
)
```

._____

SAMPLE OUTPUT

Example 1: USD to EUR

```
Enter the amount: 100
Source currency (USD/EUR/CAD): USD
Target currency (USD/EUR/CAD): EUR
100.0 USD is equal to 85.00 EUR
```

Example 2: EUR to CAD

```
Enter the amount: 200
Source currency (USD/EUR/CAD): EUR
Target currency (USD/EUR/CAD): CAD
200.0 EUR is equal to 294.00 CAD
```

Example 3: Same Currency

```
Enter the amount: 500
Source currency (USD/EUR/CAD): USD
Target currency (USD/EUR/CAD): USD
500.0 USD is equal to 500.00 USD
```

Example 4: Invalid Amount

```
Enter the amount: abc
Invalid amount
Enter the amount: -100
Invalid amount
Enter the amount: 100
```

Note: These exchange rates are predefined sample values used to demonstrate the program. They are not real-time exchange rates.

WORKING PRINCIPLE

The program follows these steps:

1. The user enters the amount.
2. The program checks whether the amount is valid.
3. The user selects the source currency.
4. The program validates the source currency.
5. The user selects the target currency.
6. The program validates the target currency.
7. The program retrieves the exchange rate from the dictionary.
8. The converted amount is calculated.
9. The result is displayed on the screen.

ADVANTAGE

1. The program is simple and easy to understand.
2. It supports three different currencies.
3. It validates the amount entered by the user.
4. It handles invalid numerical input using exception handling.
5. It prevents unsupported currency codes from being accepted.
6. It uses a dictionary to organize exchange rates.
7. It provides quick currency conversion using predefined rates.
8. It helps beginners understand fundamental Python programming concepts.

LIMITATIONS

1. The program supports only three currencies.
2. Exchange rates are predefined and may not reflect current market rates.
3. The program does not connect to a live currency exchange API.
4. It does not provide historical exchange rates.
5. It is a command-line application without a graphical user interface.

6. It does not include transaction fees or bank conversion charges.

FUTURE ENHANCEMENTS

The Currency Converter can be improved by adding the following features:

1. Support for additional currencies such as INR, GBP, JPY, and AUD.
2. Integration with a live currency exchange API.
3. Development of a graphical user interface using Tkinter.
4. Addition of a conversion history feature.
5. Improved error messages and input validation.
6. Support for repeated conversions without restarting the program.
7. Display of current exchange rates retrieved from a reliable API.

CONCLUSION

The Currency Converter project was successfully developed using Python programming. The application converts amounts between USD, EUR, and CAD using predefined exchange rates.

The project demonstrates the practical use of while loops, dictionaries, conditional statements, type conversion, and exception handling. It also shows how input validation can improve the reliability of a program.

Although the application uses predefined exchange rates, it provides a strong foundation for developing more advanced currency conversion applications. Future improvements can include live exchange rates, additional currencies, and a graphical user interface.

Overall, this mini project helped improve my understanding of Python programming and problem-solving techniques.

REFERENCES

1. Python Official Documentation – Python Programming Language.
2. Python Documentation – Built-in Functions.
3. Python Documentation – Errors and Exceptions.
4. Python Documentation – Mapping Types: Dictionary.

Programming Language: Python
Development Environment: Visual Studio Code
